

#06 Sorting & Searching

김성수

4thrunyul@gmail.com

0. 목차



1. 정렬 알고리즘

2. 탐색 알고리즘 I

- 이분 탐색
- 매개 변수 탐색

1. 정렬 알고리즘

1. 정렬 알고리즘

정렬 알고리즘(Sorting Algorithm)

- 데이터를 특정 기준(오름차순, 혹은 내림차순)에 따라 순서대로 재배열하는 작업
- 선택, 삽입, 버블 같은 $O(n^2)$ 알고리즘과 퀵, 병합 같은 $O(n \log n)$ 알고리즘이 있음.

- 정렬 전

5	1	2	4	3
---	---	---	---	---

- 오름차순 정렬

1	2	3	4	5
---	---	---	---	---

- 내림차순 정렬

5	4	3	2	1
---	---	---	---	---

1. 정렬 알고리즘



PS에서의 정렬?

- 정렬 알고리즘을 직접 구현하라는 문제는 거의 없음
- 대부분 언어에서 제공하는 내부 함수를 통해 정렬 수행
- 정렬의 종류에 대해 알고 싶다면...
- @TimoBingmann [15 Sorting Algorithms in 6 Minutes](#)

1. 정렬 알고리즘: 오름차순

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      vector<int> v = {5, 2, 4, 3, 1};
9      int arr[] = {4, 3, 1, 2, 5};
10
11      sort(v.begin(), v.end());
12      sort(arr, arr + 5);
13
14      for(int i = 0; i < v.size(); i++) {
15          cout << v[i] << " ";
16      }
17
18      cout << "\n";
19
20      for(int i = 0; i < 5; i++) {
21          cout << arr[i] << " ";
22      }
23
24      return 0;
25 }
```

sort(시작 위치, 끝 위치) | `<algorithm>` | $O(n \log n)$

- 지정한 범위에 속한 원소들을 오름차순으로 정렬
- 벡터의 경우 반복자(iterator)를, 배열의 경우 포인터를 인자로 넣어줘야 함

v.begin()

- 시작점, 벡터의 첫 번째 원소를 가리키는 반복자 반환

v.end()

- 끝점, 벡터의 마지막 원소 다음 칸을 가리키는 반복자 반환
- 만약 벡터의 크기가 n이라면 v.begin() + n는 v.end()와 같음

Input:	Copy
Expected Output:	Copy
1 2 3 4 5	
1 2 3 4 5	

1. 정렬 알고리즘: 오름차순

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  #include <string>
5
6  using namespace std;
7
8  int main() {
9      vector<string> v = {
10         "banana", "apple",
11         "pear", "watermelon"
12     };
13     string s = "GoodProgram!";
14
15     sort(v.begin(), v.end());
16     sort(s.begin(), s.end());
17
18     for(int i = 0; i < v.size(); i++) {
19         cout << v[i] << "\n";
20     }
21
22     cout << s;
23
24     return 0;
25 }
```

sort(시작 위치, 끝 위치) | `<algorithm>` | $O(n \log n)$

- vector<string>에 대한 정렬 : 사전순 정렬
- string에 대한 정렬 : ASCII Code순 정렬

Input: [Copy](#)

Expected Output: [Copy](#)

```
apple
banana
pear
watermelon
!GPAdgmooorr
```

1. 정렬 알고리즘: 내림차순

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  #include <string>
5
6  using namespace std;
7
8  int main() {
9      vector<int> v = {5, 2, 4, 3, 1};
10
11      sort(v.begin(), v.end(), greater<>());
12
13      for(int i = 0; i < v.size(); i++) {
14          cout << v[i] << " ";
15      }
16
17      cout << "\n";
18
19      string s = "Piano";
20
21      sort(s.begin(), s.end());
22
23      cout << s << "\n";
24
25      reverse(s.begin(), s.end());
26
27      cout << s;
28
29      return 0;
30 }
```

`sort(시작 위치, 끝 위치, greater<>())` | `<algorithm>` | $O(n \log n)$

- 지정한 범위에 속한 원소들을 내림차순으로 정렬

`reverse(시작 위치, 끝 위치)` | `<algorithm>` | $O(n)$

- 지정한 범위에 속한 원소들의 순서를 거꾸로 뒤집음

그 외

- 사용자 지정 함수 : 다음 챕터에서 설명
- 람다 함수 : 복잡해서 생략

```
Input:
5 2 4 3 1

Expected Output:
5 4 3 2 1
Paino
oniaP
```


1. 정렬 알고리즘: 실습

실습

- [2750 수 정렬하기](#)
 - [코드](#)
- [11931 수 정렬하기 4](#)
 - [코드](#)

1. 정렬 알고리즘

```
1  #include <iostream>
2  #include <vector>
3  #include <utility>
4  #include <string>
5  #include <algorithm>
6
7  using namespace std;
8
9  int main() {
10     vector<pair<string, int>> v = {
11         {"banana", 2},
12         {"apple", 1},
13         {"apple", 2},
14         {"pear", 1},
15         {"banana", 5}
16     };
17
18     sort(v.begin(), v.end());
19
20     for(int i = 0; i < v.size(); i++) {
21         cout << v[i].first << " ";
22         cout << v[i].second << "\n";
23     }
24
25     return 0;
26 }
```

sort(시작 위치, 끝 위치) | `<algorithm>` | $O(n \log n)$

- pair 컨테이너가 vector의 구성 요소일 때, vector를 정렬할 경우
- 우선 pair.first 원소 순으로 오름차순 정렬, 이후 pair.second 원소 순으로 오름차순 정렬을 수행

```
Input:
Expected Output:
apple 1
apple 2
banana 2
banana 5
pear 1
```

1. 정렬 알고리즘

```

1  #include <iostream>
2  #include <vector>
3  #include <utility>
4  #include <string>
5  #include <algorithm>
6
7  using namespace std;
8
9  bool compare(const pair<string, int>& a, const pair<string, int>& b) {
10     // 이름 오름차순 정렬(사전순)
11     if(a.first != b.first) {
12         return a.first < b.first;
13     }
14
15     // 개수 내림차순 정렬
16     return a.second > b.second;
17 }
18
19 int main() {
20     vector<pair<string, int>> v = {
21         {"banana", 2},
22         {"apple", 1},
23         {"apple", 2},
24         {"pear", 1},
25         {"banana", 5}
26     };
27
28     sort(v.begin(), v.end(), compare);
29
30     for(int i = 0; i < v.size(); i++) {
31         cout << v[i].first << " ";
32         cout << v[i].second << "\n";
33     }
34 }

```

sort(시작 위치, 끝 위치, 정렬 기준) | <algorithm> | $O(n \log n)$

- 만약 특정 원소는 오름차순으로, 다른 원소는 내림차순으로 정렬하고 싶다면?
- pair.first가 아니라 pair.second를 우선 정렬 기준으로 삼고 싶다면?
- 사용자 지정 정렬 함수가 필요

사용자 지정 정렬 함수

- bool compare(parameter a, parameter b)
- const : 원본 훼손 방지
- & : 인자를 복사해오는 게 아니라 주소를 지정

```

Input:                                     Copy
apple 2
apple 1
banana 5
banana 2
pear 1

Expected Output:                           Copy
apple 2
apple 1
banana 5
banana 2
pear 1

```

1. 정렬 알고리즘

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm>
5
6  using namespace std;
7
8  struct Student {
9      string name;
10     int korean;
11     int math;
12 };
13
14 bool compare(const Student& a, const Student& b) {
15     // 이름 오름차순(사전순)
16     if(a.name != b.name) {
17         return a.name < b.name;
18     }
19
20     // 국어 점수 내림차순(높은 점수 순)
21     if(a.korean != b.korean) {
22         return a.korean > b.korean;
23     }
24
25     // 수학 점수 오름차순(점수 낮은 순)
26     return a.math < b.math;
27 }
28 }
```

sort(시작 위치, 끝 위치, 정렬 기준) | `<algorithm>` | $O(n \log n)$

- 정렬 요소가 3개 이상일 때는 구조체 사용
- pair와 다르게 정렬 연산자(operator)가 정의되어있지 않으므로, 정렬 기준 함수가 무조건 필요

1. 정렬 알고리즘

```
30 int main() {  
31     vector<Student> v = {  
32         {"Bob", 90, 80},  
33         {"Eric", 100, 70},  
34         {"Alice", 90, 100},  
35         {"Alice", 70, 80},  
36         {"Eric", 100, 100},  
37     };  
38  
39     sort(v.begin(), v.end(), compare);  
40  
41     for(auto it : v) {  
42         cout << it.name << " ";  
43         cout << it.korean << " ";  
44         cout << it.math << "\n";  
45     }  
46  
47     return 0;  
48 }
```

sort(시작 위치, 끝 위치, 정렬 기준) | <algorithm> | $O(n \log n)$

- 정렬 요소가 3개 이상일 때는 구조체 사용
- pair와 다르게 정렬 연산자(operator)가 정의되어있지 않으므로, 정렬 기준 함수가 무조건 필요

Input: [Copy](#)

Expected Output: [Copy](#)

```
Alice 90 100  
Alice 70 80  
Bob 90 80  
Eric 100 70  
Eric 100 100
```

1. 정렬 알고리즘: 실습



실습

- [11651 좌표 정렬하기 2](#)
 - [코드](#)

1. 정렬 알고리즘

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <string>
5
6  using namespace std;
7
8  struct Student {
9      int score;
10     string name;
11 };
12
13 bool compare(const Student& a, const Student& b) {
14     return a.score < b.score;
15 }
16
17 int main() {
18     vector<Student> v = {
19         {5, "Bob"},
20         {3, "Eric"},
21         {5, "Alice"}
22     };
23
24     stable_sort(v.begin(), v.end(), compare);
25
26     for(int i = 0; i < v.size(); i++) {
27         cout << v[i].score << " " << v[i].name << "\n";
28     }
29 }
```

stable_sort(시작 위치, 끝 위치, 정렬 기준)

<algorithm> | $O(n \log n)$

- 특정 원소는 오름차순(혹은 내림차순)으로 정렬하되, 다른 원소는 입력 순서를 유지하고 싶다면
- ex) 왼쪽 코드는 점수순으로 오름차순 정렬하되, 점수가 같다면 입력 순서대로 출력하고 싶을 때

Input:

Copy

Expected Output:

Copy

3 Eric
5 Bob
5 Alice

1. 정렬 알고리즘: 실습



실습

- 10814 나이순 정렬
 - 코드

1. 정렬 알고리즘



Counting Sort | $O(n)$

- 원소 간의 크기를 비교하지 않고, 각 원소가 몇 번 등장했는지 개수를 세어 정렬하는 알고리즘
- 입력될 수 있는 데이터의 범위가 작을 때 유용함

1. 정렬 알고리즘

수 정렬하기 3

성공

언어 제한



시간 제한

메모리 제한

제출

정답

맞힌 사람

정답 비율

5 초 (하단 참고)

8 MB (하단 참고)

381652

93029

71240

24.188%

문제

N개의 수가 주어졌을 때, 이를 오름차순으로 정렬하는 프로그램을 작성하시오.

입력

첫째 줄에 수의 개수 N ($1 \leq N \leq 10,000,000$)이 주어진다. 둘째 줄부터 N개의 줄에는 수가 주어진다. 이 수는 10,000보다 작거나 같은 자연수이다.

출력

첫째 줄부터 N개의 줄에 오름차순으로 정렬한 결과를 한 줄에 하나씩 출력한다.

기존 문제와 비교했을 때 특이한 점?

1. 정렬 알고리즘: 실습

실습

- [10989 수 정렬하기 3](#)
 - [코드](#)

2. 탐색 알고리즘 I

2. 탐색 알고리즘의 종류

탐색 알고리즘

- 완전 탐색(브루트 포스)
- 선형 탐색
- 이분 탐색
- 삼분 탐색
- DFS, BFS
- 백트래킹

탐색 최적화 알고리즘

- 슬라이딩 윈도우(Sliding Window)
- 투 포인터(Two Pointer)
- Meet in the Middle

2. 이분 탐색



Up-Down Game

- 제가 1~100 중에 한 가지의 수를 떠올렸습니다.

2. 이분 탐색



예측?!

- 제가 떠올린 수는... 15였습니다!
- Q1 : 50 or 51
 - A1 : Down
- Q2 : 25 or 26
 - A2 : Down
- Q3 : 12 or 13
 - A3 : Up
- Q4 : 19 or 20
 - A4 : Down...

2. 이분 탐색



선형 탐색 vs 이분 탐색

- 선형 탐색 $O(n)$
- 이분 탐색 $O(\log n)$
- ex) $n = 100,000,000$ (1억)
- 선형 탐색 : 최악의 경우 1억 번 연산
- 이분 탐색 : 최악의 경우 27번 연산

이분 탐색의 전제 조건

- 탐색을 하고자 하는 데이터가 **오름차순으로 정렬**이 되어있어야 함
- 내림차순도 가능하긴 하지만, 따로 지정해야 할 것들이 많음에 따라 해당 PPT에서는 다루지 않음

2. 이분 탐색

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      int n = 5;
9      int m = 10;
10     vector<int> v = {5, 1, 4, 3, 2, 7, 6};
11
12     sort(v.begin(), v.end());
13
14     cout << binary_search(v.begin(), v.end(), n);
15     cout << "\n";
16     cout << binary_search(v.begin(), v.end(), m);
17
18     return 0;
19 }
```

Input:

Copy

Expected Output:

Copy

1
0

binary_search(시작 위치, 끝 위치, 찾을 값)

<algorithm> | $O(\log n)$

- 지정한 범위에 속에 찾으려는 값 존재 여부를 bool 타입으로 반환
- 값이 존재한다면 true(1), 존재하지 않는다면 false(0) 반환
- 벡터의 경우 반복자(iterator)를, 배열의 경우 포인터를 인자로 넣어줘야 함

2. 이분 탐색

```

1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      int n = 4;
9      int m = 10;
10     vector<int> v = {1, 2, 3, 4, 5, 6, 7};
11
12     // 4 이상의 값이 처음 나오는 위치
13     // (값: 4, 인덱스: 3)
14     cout << lower_bound(v.begin(), v.end(), n) - v.begin();
15     cout << "\n";
16
17     // 10 이상의 값이 처음 나오는 위치
18     // (없으므로 v.end()인 인덱스: 7)
19     cout << lower_bound(v.begin(), v.end(), m) - v.begin();
20     cout << "\n";
21
22     // 4를 초과하는 값이 처음 나오는 위치
23     // (값: 5, 인덱스: 4)
24     cout << upper_bound(v.begin(), v.end(), n) - v.begin();
25     cout << "\n";
26
27     // 10을 초과하는 값이 처음 나오는 위치
28     // (없으므로 v.end()인 인덱스: 7)
29     cout << upper_bound(v.begin(), v.end(), m) - v.begin();
30
31     return 0;
32 }

```

lower_bound(시작 위치, 끝 위치, 찾을 값)

<algorithm> | $O(\log n)$

- 지정한 범위 내에서 찾을 값 이상의 값이 처음으로 나타나는 위치를 반환
- 조건에 맞는 원소가 없으면 끝 위치를(v.end()) 반환
- 반환형은 반복자(iterator) 또는 포인터이므로, 인덱스를 구하려면 시작 위치를 빼줘야 함

upper_bound(시작 위치, 끝 위치, 찾을 값)

<algorithm> | $O(\log n)$

- 지정한 범위 내에서 찾을 값 초과하는 값이 처음으로 나타나는 위치를 반환
- 조건에 맞는 원소가 없으면 끝 위치를(v.end()) 반환
- 두 함수의 반환 위치 차이(upper_bound - lower_bound)를 통해 vector나 array이 포함하는 특정 원소의 개수를 빠르게 구할 수 있음

Input:	Copy
Expected Output:	Copy
3	
7	
4	
7	

2. 이분 탐색

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      int target = 3;
9      int cnt = 0;
10     vector<int> v = {1, 2, 3, 4, 5, 6, 7};
11
12     int low = 0;
13     int high = v.size() - 1;
14     int result = -1;
15
16     while (low <= high) {
17         ++cnt;
18
19         int mid = (low + high) / 2;
20
21         if (v[mid] == target) {
22             result = mid;
23             break;
24         }
25         else if (v[mid] > target) high = mid - 1;
26         else low = mid + 1;
27     }
28
29     cout << "index: " << result;
30     cout << "\ncalc: " << cnt;
31
32     return 0;
33 }
```

Input: [Copy](#)

Expected Output: [Copy](#)

index: 2
calc: 3

2. 이분탐색: 실습



실습

- 1920 수 찾기
 - 코드
- 10816 숫자 카드 2
 - 코드

2. 이분 탐색



매개변수 탐색(Parametric Search)

- 문제에서 요구하는 최적화 문제(최댓값이나 최솟값을 찾는 문제)를 결정 문제로 바꾸어 풀이하는 기법
- ex) 16층인 건물이 있다. n층에서 계란을 던졌을 때 계란이 깨지지 않는 최대 층을 구하세요.
- Case1 : 1~5층 안깨짐, 6~16층 깨짐
 - A. 8층 : 깨짐
 - B. 4층 : 안깨짐
 - C. 6층 : 깨짐
 - D. 5층 : 안깨짐
 - => 경계값을 찾아냈기 때문에 5층이 최대층인 것을 알 수 있음
- Case2 : 1~10층 안깨짐, 11~16층 깨짐
 - A. 8층 : 안깨짐
 - B. 12층 : 깨짐
 - C. 10층 : 안깨짐
 - D. 11층 : 깨짐
 - => 경계값을 찾아냈기 때문에 10층이 최대층인 것을 알 수 있음

2. 이분 탐색



매개변수 탐색의 조건

- 정답의 범위가 연속적이거나 고정적이어야 함
- 단조성이 있어야 함
 - 앞서 건물에서 계란을 떨어뜨리는 문제에 성립할 수 있었던 이유
 - n 층에서 깨짐 $\Rightarrow n$ 이상의 층에서도 계란이 깨지구나
 - n 층에서 안깨짐 $\Rightarrow n$ 이하의 층에서도 계란이 안깨지구나

매개변수 탐색이 어려운 이유?

- 매개변수 탐색을 통해 푸는 문제인지 직관적으로 확인하기 어려움
- 그리디 알고리즘, DP와 함께 엮이는 경우도 있기 때문에 발상이 힘들
- 잘못 구현할 경우 경계값 처리에서 무한루프에 빠질 수 있음

2. 이분탐색: 실습



실습

- [2805 나무 자르기](#)
 - [코드](#)